Part 1: Academic Forensic

Question 1: The Initialization Penalty

In Snippet A, the member `name` is default-constructed first. Then, inside the constructor body, the assignment operator copies the value of `n` into `name`. This can cause an unnecessary extra operation.

In Snippet B, `name` is initialized directly using the member initializer list. This avoids default construction followed by assignment and is generally more efficient.

Snippet B is considered professional C++ practice because it initializes members directly. It is also necessary for members that cannot be default-constructed, such as references and certain `const` members.

Question 2: The Rule of Three Trigger

The Rule of Three states that if a class requires a custom destructor to manage a resource, it will often also require a custom copy constructor and copy assignment operator.

A file handle is a resource that should have clear ownership. A shallow copy duplicates the handle or pointer value instead of creating an independent, safely managed resource. This can cause multiple objects to manage the same resource, leading to double closing, invalid access, or other resource-management errors.

A custom copy constructor must define a safe copying strategy. Depending on the resource, the class may need to duplicate the underlying resource or prohibit copying entirely.

Question 3: Move vs. Copy

A Copy Constructor creates a new object by copying the source object's data. When the class owns dynamically allocated memory, a deep copy creates a separate allocation so that both objects can manage their resources independently.

A Move Constructor transfers ownership of resources from the source object to the newly created object, usually by transferring a pointer instead of copying the entire data. It commonly uses an rvalue reference (`&&`).

After transferring ownership, the source pointer should be set to `nullptr`, and the source object should remain in a valid state. This prevents both objects from attempting to release the same resource.

Part 2: Code Review & Sabotage

Snippet A: The Dangling Reference

The problem is that `connect()` receives its argument by value, so a copy of `myConn` is created. If the class uses the compiler-generated copy constructor, the pointer `port` is copied as an address, not as a separate allocation. Both objects then point to the same dynamically allocated integer.

If the class destructor deletes `port`, the temporary copy may release the memory when `connect()` finishes. The original object is then left with a dangling pointer, and its later destruction attempts to release the same memory again, causing undefined behavior such as a double-free crash.

The class should implement a safe copy constructor and copy assignment operator, or disable copying. Alternatively, it can use an appropriate smart pointer or a reference parameter if copying is unnecessary.

Note: As displayed in the screenshot, the class does not show a destructor. Without a destructor that releases `port`, the exact code shown would leak memory rather than necessarily crash. The double-free explanation applies if the intended destructor deletes `port`.

Snippet B: The Abstraction Leak

The class violates encapsulation because `getPassword()` returns a mutable `char*` pointing directly to the internal password buffer. External code can modify the password without using a controlled class interface.

Two lines that demonstrate the problem are:

```
PasswordManager manager("original");
std::strcpy(manager.getPassword(), "changed");
```

This changes the internal password directly, bypassing the class's intended interface.

To fix the issue, return a `const char*` instead of a mutable `char*`, and make the getter a `const` member function:

```
const char* getPassword() const {
    return decrypted_password;
}
```

This prevents callers from modifying the buffer through the returned pointer. If password changes are required, provide a separate setter that validates the new value and manages the buffer safely.